

Adam: A Method for Stochastic Optimization

1. Introduction

1.1 연구 배경

- **목적 함수의 특징** : 딥러닝 손실 함수는 미니배치와 Dropout 등으로 인해 매 스텝 그래디언트가 달라지는 **확률적(Stochastic)** 성격을 가짐
- **기존 방식의 한계**
 - SGD : 빠르지만 방향과 스텝이 불안정함.
 - 2차 미분 기반 : Hessian 역행렬 계산 비용($O(d^2 \sim d^3)$)이 커서 대규모 파라미터에는 비현실적임.

1.2 옵티마이저 계보

- GD $\rightarrow$ SGD 로 속도는 개선되었으나 수렴 불안정
- SGD 를 보완한 방법들
 - **방향성** : Momentum(1964), NAG(1983) — 관성으로 진동을 줄이고 가속
 - **스텝 사이즈** : AdaGrad(2011), RMSProp(2012) — 파라미터별 학습률을 적응적으로 조절
 - **Adam(2014)**은 이 두 갈래를 통합

1.3 논문의 기여

- ① 구현이 간단하고 연산이 효율적이며 메모리 요구량이 거의 없음
- ② 그래디언트의 대각 재스케일링(diagonal rescaling)에 영향받지 않음 $\rightarrow$ 파라미터별 학습률 자동 조정
- ③ 데이터 또는 파라미터가 큰 대규모 문제에 적합
- ④ 잡음(noisy)·희소(sparse) 그래디언트에 강건
- ⑤ 하이퍼파라미터(β_1, β_2)가 직관적이고 튜닝이 거의 불필요
- ⑥ 실험에서 SGD·AdaGrad·RMSProp 대비 일관된 우수 성능

2. Related Work

2.1 선행 연구

- **AdaGrad**
 - 파라미터별 그래디언트 제곱을 누적($h \leftarrow h + g \odot g$)하여 학습률을 자동 조절하는 적응형 옵티마이저
 - 큰 그래디언트가 누적된 파라미터는 학습률이 줄고, 작은 그래디언트만 받은 파라미터는 학습률이 크게 유지되어 **sparse gradient**에 적합
 - 단, 무한 누적으로 학습률이 0에 수렴하는 한계가 있음.
- **RMSProp**
 - AdaGrad의 학습률 소실을 **지수 이동 평균(EMA)**으로 해결
 - 이전의 기울기는 ρ (decaying rate)만큼 반영하고, 현재의 기울기는 $(1 - \rho)$ 만큼 반영하여, 먼 과거의 기울기는 지수적으로 점차 잊고 새로운 곡률에 적응
 - AdaGrad의 학습률 0 수렴 문제 해소 $\rightarrow$ Adam의 2차 moment 추정에 그대로 계승됨

2.2 Adam의 차별성

- **Adam(Adaptive Moment Estimation)** : 적은 연산량을 지닌 first-order gradient 기반 stochastic optimization 알고리즘

- 1차 moment(그래디언트 평균 → 방향)와 2차 moment(그래디언트 제곱 평균 → 크기)를 동시에 추정하여 방향 + 스텝 사이즈를 함께 적응 조절
- Momentum + RMSProp + AdaGrad의 장점을 하나의 알고리즘으로 통합하고, 편향 보정을 추가하여 초기 학습 안정성까지 확보

3. Algorithm

3.1 Adam 알고리즘

- 초기화: $m_0 = 0, v_0 = 0 \rightarrow \theta$ 수렴 시까지 반복:
 - ① 그래디언트 계산: $g_t = \nabla \theta f_t(\theta_{t-1})$
 - ② 모멘트 갱신: $m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t, v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$
 - ③ 편향 보정: $\hat{m}_t = m_t / (1 - \beta_1^t), \hat{v}_t = v_t / (1 - \beta_2^t)$
 - ④ 업데이트: $\theta_t = \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$
- 기본값: $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$

3.2 편향 보정의 필요성

- $m_0 = v_0 = 0$ 초기화로 인해 초기 추정치가 0으로 편향되어 실제값보다 $(1 - \beta_1^t)$ 배만큼 작게 추정됨
 $\rightarrow (1 - \beta_1^t)$ 로 나누어 보정하면 초기부터 안정적 수렴 가능

3.3 유효 학습률 & SNR 해석

- $\frac{\hat{m}_t}{\sqrt{\hat{v}_t}}$ 신호 대 잡음비(SNR)로 해석됨
 - SNR $\uparrow$ (일관된 그래디언트) $\rightarrow$ 큰 스텝 / SNR $\downarrow$ (잡음 큼) $\rightarrow$ 작은 스텝
- 최적값 근처에서 SNR이 자연스럽게 작아져 자동 어닐링 효과 — LR 스케줄 불필요

3.4 Step Size 상한

- $\|\Delta_t\| \leq \alpha$: α 가 "파라미터 공간의 trust region" 역할 $\rightarrow$ 튜닝이 직관적

3.5 효율적 구현 형태

- 편향 보정을 학습률에 흡수: $\alpha_t = \alpha \cdot \frac{\sqrt{1 - \beta_2^t}}{(1 - \beta_1^t)}$ 로 나눴셈을 기존 2회에서 1회로 단축하면, $\hat{m}, \hat{v}$ 를 별도 저장할 필요 없어 메모리 절감

4. Experiments

다양한 환경에서 Adam을 기존 옵티마이저들과 비교:

- **MNIST (Logistic Regression)**
 - Adam의 수렴 속도가 SGD와 유사하며, AdaGrad보다 빠름 $\rightarrow$ dense feature 환경에서도 Adam이 안정적으로 동작함
- **IMDB (BoW + 50% dropout)**
 - Sparse feature 환경에서도 Adam이 SGD보다 빠른 수렴
- **MLP (2-hidden $\times$ 1000, ReLU + dropout)**

- Adam 이 다른 옵티마이저에 비해 가장 낮은 training cost 로 빠르게 수렴하며, 2 차 근사 기반 SFO 보다도 iterations·walltime 모두 우위를 가짐
- CNN / CIFAR-10 (C64-C64-C128-FC)
 - Adam 은 단기(3ep)·장기(45ep) 모두 안정적으로 가장 낮은 cost 에 도달
- Bias Correction 실험
 - β_2 가 1 에 가까울수록 초기화 편향이 커져 초반 학습 불안정 → bias correction 은 학습 초반 안정성에 필수적

5. Ablation Studies

5.1 AdaMax

- Adam 의 $v_t(L^2\text{-norm})$ 를 $L^\infty\text{-norm}(\text{max norm})$ 으로 대체 : $u_t = \max(\beta_2 \cdot u_{t-1}, |g_t|)$
- 분모가 0 이 될 수 없어 ϵ 불필요, v_t 에 대한 bias correction 도 불필요하다는 장점을 가짐
 - 권장 기본값 : $\alpha=0.002, \beta_1=0.9, \beta_2=0.999$

5.2 Temporal Averaging

- Polyak-Ruppert averaging(모든 θ_t 평균)이 일반화에 유리하나 메모리·계산 비용이 크다는 한계
 - 지수가중 이동평균 $\theta_t = \beta_2 \cdot \theta_{t-1} + (1 - \beta_2) \cdot \theta_t$ 로 변수 1 개만 추가하여 수렴 안정성과 일반화 성능을 향상 / Adam·AdaMax 에 결합 가능.

6. Conclusion

- Adam 의 강점
 - 효율성 : $O(d)$ 메모리·계산
 - 스케일 불변성 : Gradient 그래디언트 대각 재스케일링에 견고
 - 직관적 HP : 기본값($\alpha=0.001, \beta_1=0.9, \beta_2=0.999$)으로 대부분 작동
- Momentum + RMSProp 의 장점을 통합하고, 편향 보정으로 초기 안정성까지 확보
- Adam 의 영향력 : BERT·GPT·ResNet·GAN 등 현대 딥러닝의 사실상 표준 옵티마이저 (120,000+ Citations)

Efficient Memory Management for Large Language Model Serving with PagedAttention

1. Introduction

1.1 연구 배경

- **목표** : LLM Serving 시스템에서 KV cache 메모리를 효율적으로 관리하여 throughput 을 높이는 것
- **LLM 동작 방식**
 - Autoregressive Transformer 기반으로 token을 순차 생성하며, 매 step마다 이전 모든 token의 key·value vector(KV cache)가 필요
 - Sequential 생성 구조로 인해 workload가 memory-bound이므로 GPU 연산 활용이 낮고 throughput이 제한됨

1.2 기존 연구의 한계

- 기존 시스템(FasterTransformer, Orca)은 KV cache 를 **연속된 메모리(contiguous memory)**에 저장 → 두 가지 비효율 발생 :
 - **Fragmentation** : 최대 길이 기준으로 미리 할당 → internal fragmentation / 요청마다 크기가 다름 → external fragmentation
 - **Memory Sharing 불가** : parallel sampling, beam search 등에서 contiguous 저장 방식으로는 KV cache 공유 불가

1.3 논문의 Contributions

OS의 **virtual memory & paging**에서 영감을 받은 **PagedAttention** + LLM serving engine **vLLM**

- Near-zero waste in KV cache memory
- Flexible sharing of KV cache within and across requests

2. Background

- **KV Cache**
 - LLM이 token을 생성할 때, 이전 모든 token의 key·value vector를 재계산하지 않고 저장해두는 것
 - **Prompt phase**(전체 입력을 한 번에 처리, GPU 효율적)와 **Autoregressive generation phase**(token 1개씩 순차 생성, memory-bound)로 나뉨
- **Batching**
 - 여러 요청을 묶어 처리하면 model weights 이동 overhead가 분산되어 throughput 향상
 - **Iteration-level scheduling**으로 매 iteration마다 완료된 요청 제거 + 새 요청 추가하여 fine-grained batching 가능

3. Memory Challenges in LLM Serving

- **Large KV cache** : 요청 수가 늘수록 KV cache 크기가 급격히 증가하여 메모리가 bottleneck
- **Complex decoding** : parallel sampling, beam search 등에서 KV cache sharing 기회를 기존 시스템이 활용하지 못함
- **Unknown output lengths** : 출력 길이를 사전에 알 수 없어 메모리 부족 시 일부 요청의 KV cache를 삭제하거나 swap out하는 scheduling 결정 필요

4. Method

4.1 PagedAttention

- **핵심 아이디어**
 - OS의 virtual memory & paging 개념을 KV cache 관리에 적용(blocks = pages, tokens = bytes, requests = processes)
 - 각 sequence의 KV cache를 고정 크기의 KV block으로 분할하고, block들을 non-contiguous physical memory에 저장
- **효과**
 - internal fragmentation → 작은 block + on-demand 할당으로 완화
 - external fragmentation → 모든 block이 동일 크기이므로 제거
 - memory sharing → block 단위로 가능

4.2 KV Cache Manager

- Block Table을 통해 logical KV blocks → physical KV blocks 매핑을 관리
- **효과** : 메모리 낭비 최소화(이전 block이 가득 찼을 때만 새 block을 할당하므로, 요청당 낭비는 마지막 block 하나 이내로 제한 / 요청 완료 시 KV block을 즉시 해제하여 다른 요청이 재사용)

4.3 다양한 Decoding 시나리오 적용

- **Parallel Sampling** : prompt KV cache는 하나의 physical block에 저장해 공유하고, 각 block의 reference count로 참조 횟수를 추적하며 마지막 logical block만 copy-on-write로 처리
- **Beam Search** : prompt block뿐 아니라 중간 block도 candidate 간 공유 가능하며, sharing pattern이 디코딩 진행에 따라 동적으로 변화
- **Shared Prefix** : predefined shared prefix의 physical block을 미리 예약하고, 해당 prefix를 포함한 요청은 logical block을 캐싱된 physical block에 직접 매핑하여 prefix 계산 중복 제거

4.4 Scheduling & Preemption & Distributed Execution

- **Scheduling** : FCFS(First-Come-First-Serve)로 공정성 보장
- **Preemption 정책** : GPU physical block 부족 시, All-or-nothing 정책에 따라 한 sequence의 block을 전부 evict하거나 전부 유지
- **Distributed Execution** : Megatron-LM 방식의 tensor model parallelism 지원하며, 단일 KV cache

manager를 centralized scheduler 내에 두어 각 GPU worker가 동일한 block mapping을 공유

5. Conclusion

- LLM serving에서 KV cache 메모리 낭비 문제를 식별·정량화하고, OS paging에서 영감을 받은 **PagedAttention**을 제안
- vLLM 구현을 통해 near-zero waste + flexible KV cache sharing을 달성
- **실험 결과** : FasterTransformer, Orca 대비 **2~4 × throughput 향상** (모델 정확도 변화 없음)
→ 긴 sequence, 큰 모델, 복잡한 decoding algorithm일수록 개선 효과가 더 크게 나타남